



PROYECTO DE INGENIERÍA DE DATOS

Modern Data Stack para una Cooperativa Olivarera

Data Warehouse · dbt · PostgreSQL · Airflow · Docker

Versión	v1.0 — Borrador Inicial
Estado	En desarrollo
Stack principal	Python · dbt · PostgreSQL · Airflow · Docker
Sector	Agroindustria — Producción de Aceite de Oliva

1. Contexto y Justificación

Las cooperativas olivereras son organizaciones con una cadena de valor compleja que abarca desde la gestión agrícola de parcelas y socios hasta la transformación industrial del fruto y la comercialización del aceite resultante. Esta complejidad genera un volumen considerable de datos operativos que, en la mayoría de casos, permanecen dispersos en sistemas heterogéneos: hojas de cálculo Excel, ERPs locales y aplicaciones internas sin integración.

La consecuencia directa es la imposibilidad de obtener una visión unificada y fiable del negocio. Los responsables operativos y de dirección no pueden responder con agilidad a preguntas críticas como el rendimiento por parcela, la rentabilidad por socio o la evolución de la producción entre campañas.

Áreas de gestión afectadas

El proyecto centraliza y estructura la información de las siguientes áreas de negocio:

- Gestión de socios y sus datos identificativos
- Registro y caracterización de parcelas agrícolas
- Entregas de aceituna en báscula (pesaje y calidad)
- Procesos de molturación y extracción de aceite
- Análisis de calidad del aceite producido
- Liquidaciones económicas y pagos a socios
- Ventas y comercialización del producto final

OBJETIVO

Construir un Data Warehouse moderno, reproducible y bien documentado que centralice toda la información operativa de la cooperativa, permitiendo análisis fiables y toma de decisiones basada en datos.

2. Objetivo del Proyecto

El objetivo principal es diseñar e implementar una plataforma analítica end-to-end que cubra todo el ciclo del dato: desde su generación sintética con reglas de negocio realistas hasta su exposición en un modelo dimensional listo para consumo analítico.

Entregables principales

#	Entregable	Descripción
01	Datos sintéticos realistas	Generador configurable con semilla para reproducibilidad, simulando campañas reales
02	Ingesta en PostgreSQL	Carga automatizada en el esquema RAW desde múltiples fuentes simuladas

03	Transformaciones con dbt	Modelos SQL versionados, testeados y documentados mediante dbt Core
04	Modelo estrella (star schema)	Dimensiones y tablas de hechos optimizadas para análisis dimensional
05	Tests de calidad de datos	Validaciones automáticas de integridad, unicidad y rangos en dbt
06	Orquestación con Airflow	DAG que automatiza el pipeline completo de extremo a extremo
07	Entorno Docker reproducible	docker-compose que levanta toda la infraestructura con un solo comando
08	Documentación profesional	README, diagramas de arquitectura y modelo de datos listos para GitHub

3. Preguntas de Negocio a Responder

El Data Warehouse está diseñado para habilitar respuestas directas a las siguientes preguntas operativas y estratégicas. Cada una corresponde a un KPI o análisis implementable sobre el modelo estrella final.

Pregunta de negocio		Tabla de hechos
	¿Cuántos kilogramos entrega cada socio por campaña?	fact_deliveries
	¿Qué parcelas tienen mejor rendimiento graso?	fact_milling_batches
	¿Cómo influye la calidad del fruto en el pago final al socio?	fact_settlements
	¿Cómo evoluciona la producción entre campañas agrícolas?	fact_deliveries / fact_milling_batches
	¿Qué socios resultan más rentables para la cooperativa?	fact_settlements
	¿Cuál es la estacionalidad de las entregas durante la campaña?	fact_deliveries + dim_date
	¿Cómo afecta la variedad de aceituna al rendimiento final?	fact_milling_batches + dim_plot

4. Arquitectura General

El sistema adopta una arquitectura Modern Data Stack basada en componentes open-source bien establecidos. Todos los servicios se ejecutan en contenedores Docker, lo que garantiza paridad entre entornos de desarrollo, prueba y producción.

Infraestructura tecnológica

Componente	Tecnología	Función en el sistema
Orquestación	Apache Airflow	Automatiza y monitoriza el pipeline completo mediante DAGs
Base de datos	PostgreSQL 15	Almacena las tres capas del DWH: RAW, STAGING y MARTS
Transformaciones	dbt Core	Transforma y documenta los datos mediante SQL versionado
Generación e ingesta	Python 3.11	Genera los datos sintéticos y los carga en el esquema RAW
Contenedores	Docker Compose	Levanta todo el entorno con un único comando
Visualización (opc.)	Metabase	Dashboards y exploración de datos para negocio

Capas del Data Warehouse

El modelo de datos se organiza en tres capas diferenciadas que reflejan el nivel de procesamiento y transformación de los datos:

Capa	Schema PostgreSQL	Descripción
RAW	raw	Datos tal como llegan al sistema. Sin transformaciones ni limpieza. Fuente de verdad inmutable.
STAGING	staging	Datos limpios, tipados y normalizados. Modelos stg_* generados por dbt. Sin joins entre entidades.
MARTS	marts	Modelo estrella listo para análisis. Dimensiones enriquecidas y tablas de hechos con KPIs calculados.

5. Modelo de Datos

El modelo sigue la arquitectura dimensional clásica (Kimball), compuesta por dimensiones que describen el contexto del negocio y tablas de hechos que registran los eventos cuantificables.

Dimensiones

Tabla	Descripción	Atributos clave
dim_date	Dimensión de tiempo	fecha, día, semana, mes, trimestre, campaña, es_festivo
dim_campaign	Campaña agrícola	año_inicio, año_fin, nombre_campaña, estado
dim_member	Socios de la cooperativa	nombre, NIF, municipio, fecha_alta, estado

dim_plot	Parcelas agrícolas	referencia_catastral, municipio, superficie_ha, variedad, sistema_riego
dim_quality_grade	Grados de calidad del aceite	categoría (virgen extra, virgen, lampante), acidity_max, descripción
dim_product *	Tipos de producto final	descripción, formato, denominación de origen
dim_customer *	Clientes compradores	razón social, país, canal de venta

* Entidades opcionales para fases avanzadas del proyecto.

Tablas de Hechos

Tabla	Granularidad	Métricas principales
fact_deliveries	1 fila = 1 entrega	kg_brutos, kg_netos, humedad_pct, impurezas_pct, rendimiento_teorico
fact_milling_batches	1 fila = 1 lote	kg_aceituna_procesada, litros_aceite, rendimiento_real_pct, horas_proceso
fact_settlements	1 fila = 1 liquidación por socio/campaña	importe_base, penalizaciones, bonificaciones, importe_final, precio_kg
fact_sales *	1 fila = 1 línea de venta	kg_vendidos, precio_unitario, importe_total, margen

6. Generación de Datos Sintéticos

El proyecto no depende de datos reales de ninguna cooperativa. Todo el conjunto de datos se genera artificialmente mediante un módulo Python parametrizable, diseñado para simular con fidelidad los patrones operativos de una campaña olivarera.

Parámetros de escala configurables

Entidad	Volumen	Notas
Campañas agrícolas	3 a 5	Historial de varios años para análisis de tendencias
Socios registrados	200 a 500	Distribución realista por tamaño de explotación
Parcelas	800 a 2.000	Varias parcelas por socio, con variedad y coordenadas
Entregas de aceituna	Hasta millones	Escalable según necesidad de pruebas de rendimiento

Reglas de negocio en el generador

- Estacionalidad realista: concentración de entregas en noviembre y diciembre
- Rendimiento graso correlacionado con la variedad de aceituna

- Penalizaciones automáticas por exceso de humedad o impurezas
- Coherencia referencial garantizada entre todas las entidades
- Semilla configurable (seed) para asegurar reproducibilidad total de los experimentos

CONFIG

El generador acepta un fichero YAML de configuración donde se definen la escala, la semilla aleatoria y los parámetros de calidad. Esto permite ejecutar el mismo pipeline con datasets de diferente tamaño sin modificar el código.

7. Pipeline de Datos

El pipeline sigue un flujo lineal de cinco etapas que transforma los datos brutos generados en modelos analíticos listos para consumo. Airflow orquesta la ejecución garantizando el orden de dependencias y el registro de cada ejecución.

Paso	Etapas	Herramienta	Salida esperada
01	Generación de datos sintéticos	Python + Faker + NumPy	Ficheros CSV/JSON en data/raw/
02	Carga al esquema RAW	Python + SQLAlchemy	Tablas pobladas en schema raw de PostgreSQL
03	Limpieza y normalización (Staging)	dbt — modelos stg_*	Tablas limpias en schema staging
04	Construcción del modelo estrella	dbt — dimensiones y hechos	Tablas dim_* y fact_* en schema marts
05	Tests de calidad de datos	dbt test + Great Expectations	Informe de validación con estado pass/fail

8. Plan de Trabajo por Milestones

El proyecto se divide en seis milestones iterativos, cada uno con entregables concretos y verificables. Este esquema permite validar el avance de forma incremental y facilita la revisión por parte de mentores o compañeros.

Milestone	Nombre	Entregables clave
M0 · Entorno	Infraestructura base	Docker Compose operativo, PostgreSQL accesible, estructura del repositorio creada
M1 · RAW	Datos en crudo	Generador Python funcional, tablas raw pobladas, log de ingestas implementado

M2 · Staging	Datos normalizados	Modelos stg_* en dbt, tests básicos pasando, tipado y nombres normalizados
M3 · Marts	Modelo dimensional	Dimensiones y hechos contruidos, KPIs calculados disponibles para consulta
M4 · Orquestación	Pipeline automatizado	DAG en Airflow ejecutando el pipeline completo de extremo a extremo
M5 · Cierre	Documentación y presentación	README profesional, diagrama de arquitectura, diagrama ERD del modelo estrella

9. Estructura del Repositorio

La organización del repositorio refleja la separación de responsabilidades entre los distintos componentes del sistema. Cada directorio tiene un propósito único y bien definido.

Ruta	Contenido y propósito
<code>olive-coop-dwh/</code>	Raíz del proyecto. README.md, Makefile, .gitignore, .env.example
├─ <code>docker/</code>	Dockerfiles y configuración de todos los servicios (Airflow, dbt, app Python)
└─ <code>airflow/</code>	Imagen Docker personalizada para Airflow con dependencias del proyecto
└─ <code>dbt/</code>	Imagen Docker para dbt con perfil de conexión a PostgreSQL
├─ <code>airflow/</code>	DAGs de Airflow que orquestan el pipeline completo
└─ <code> dags/</code>	Fichero(s) DAG con las tareas del pipeline y sus dependencias
├─ <code>dbt/</code>	Proyecto dbt completo (modelos, tests, macros, seeds, documentación)
└─ <code>models/</code>	Subdirectorios staging/ y marts/ con los modelos SQL
└─ <code>profiles/</code>	profiles.yml que lee credenciales de variables de entorno
├─ <code>src/</code>	Código Python: generador de datos e ingesta al esquema RAW
└─ <code>ingestion/</code>	Scripts de carga a PostgreSQL con registro de ingestas
└─ <code>generator/</code>	Generador de datos sintéticos configurable
├─ <code>data/</code>	Datos locales (excluidos de Git). Subdirectorios raw/ y samples/
├─ <code>docs/</code>	Diagramas de arquitectura y ERD del modelo de datos
<code>.env.example</code>	Plantilla pública con todas las variables de entorno necesarias
<code>Makefile</code>	Atajos para los comandos más habituales del proyecto

10. Métricas de Éxito y Buenas Prácticas

Criterios de éxito del proyecto

El proyecto se considerará completo y exitoso cuando cumpla los siguientes criterios objetivos:

- El pipeline se puede reproducir desde cero con un único comando (make dev)
- El modelo estrella está completo, documentado y las relaciones son correctas
- Todos los tests de dbt pasan sin errores en el dataset de referencia
- Las cargas incrementales están implementadas y verificadas
- Los KPIs de negocio son consultables directamente en el schema marts
- La documentación permite que un tercero comprenda y ejecute el proyecto sin asistencia

Buenas prácticas de desarrollo

Práctica	Implementación
Control de versiones	Git con ramas feature/*, hotfix/* y merge requests revisados
Commits semánticos	Prefijos feat:, fix:, test:, docs:, refactor:, chore:
Seguridad	Credenciales solo en .env (excluido de Git). Plantilla en .env.example
Reproducibilidad	Docker Compose + seed fija en el generador garantizan paridad de entornos
Calidad de datos	Tests dbt en cada capa: not_null, unique, accepted_values, relationships
Documentación	Cada modelo dbt con descripción y columnas documentadas en schema.yml

11. Futuras Mejoras y Evolución

Una vez consolidada la versión inicial, el proyecto puede evolucionar en las siguientes direcciones para aumentar su complejidad técnica y valor como pieza de portfolio:

Corto plazo

- Integración con datos climáticos reales (temperatura, precipitación) via API pública
- Particionado de tablas de hechos por campaña para mejorar el rendimiento en consultas
- Implementación de snapshots SCD Tipo 2 en dbt para historificar dimensiones cambiantes

Medio plazo

- Pipeline CI/CD con GitHub Actions: tests automáticos en cada Pull Request
- Benchmark de rendimiento con datasets de varias decenas de millones de filas
- Dashboard en Metabase con los KPIs de negocio principales

Largo plazo

- Migración de PostgreSQL a un OLAP columnar (DuckDB, ClickHouse) para analítica a escala
- Ingesta de datos reales de cooperativas piloto con anonimización
- Modelo de predicción de rendimiento graso usando las variables de calidad disponibles

12. Resultado Final Esperado

Al completar el proyecto, se dispondrá de un repositorio GitHub de calidad profesional que demuestre competencias en el diseño e implementación de una plataforma de datos moderna de extremo a extremo.

PORTFOLIO

Este proyecto está diseñado específicamente como pieza central de un portfolio orientado a roles de Data Engineer. Cubre el stack tecnológico más demandado en el mercado y demuestra capacidad para trabajar con datos a escala en entornos reproducibles.

Competencias técnicas demostradas

Área	Competencias específicas
Modelado dimensional	Diseño de esquema estrella, definición de granularidades, elección de métricas y atributos
SQL avanzado	CTEs, window functions, agregaciones condicionales, optimización de consultas analíticas
dbt Core	Modelos por capas, tests de calidad, macros Jinja, documentación y linaje de datos
Apache Airflow	Diseño de DAGs, gestión de dependencias, manejo de errores y reintentos
Docker & DevOps	Dockerfiles multi-stage, Compose, variables de entorno, volúmenes persistentes
Python para datos	Generación sintética, ingesta con SQLAlchemy, logging, configuración YAML
Documentación técnica	README estructurado, diagramas de arquitectura, documentación de modelos